Лабораторная работа №4

Реализация основных моделей в агентном подходе

Люпп Софья Романовна

Содержание

1 Цель работы **5**

2 Задание **6**

3 Теоретическое введение **7**

4 Выполнение лабораторной работы **9**

4.1 Сканирование коэффициента заразности β 25

5 Выводы **55**

Список литературы **56**

Список иллюстраций

4.1 Создание файла sir_model.jl 9
4.2 Создание скрипта sir_run_basic.jl 9
4.3 Создание скрипта sir_scan_beta.jl 10
4.4 Создание sir_migration_effect.jl 10
4.5 sir_optimize_parameters.jl 10
4.6 Distributions 11
4.7 Статистика по количеству S, I, R 11
4.8 Запуск эксперимента 12
4.9 Доля инфицированных от количества заразности 12
4.10 Эксперименты с миграцией 13
4.11 Численность людей в пике от интенсивности миграции 13
4.12 migration_scan_all.cvs 14
4.13 beta_scan_all.cvs 15
4.14 BlackBoxOptim 15
4.15 Вывод файла sir_visualize_dynamics.jl в консоль 16
4.16 Итоговый отчет comprehensive_analysis.png 17
4.17 Полученные выводы в консоль для заданий 1-5 18
4.18 Вывод в консоль задания 6 18
4.19 Получившийся график задания 1 19
4.20 График задания 3 19
4.21 График задания 4 20
4.22 График задания 5 20

Список таблиц

1 Цель работы

Целью лабораторной работы является знакомство с агентным подходом и реализация основных моделей в агентном подходе на примере модели SIR.

2 Задание

1. Написать скрипты для модели SIR;
2. Запустите модель с параметрами по умолчанию, определив базовое репродуктивное число R_0;
3. Исследовать минимальное значение ⊠, при котором возникает эпидемия и сравнить с теоретическим порогом;
4. Реализовать эффект гетерогенности, задав разные значения ⊠ для разных городов;
5. Исследовать влияние интенсивности миграции на скорость распространения инфекции из одного города в другие;
6. Модифицировать модель, добавив возможность закрытия города (обнуление миграции из него) при превышении порога заболеваемости;
7. Используя оптимизацию, найти параметры, которые минимизируют общее число умерших при сохранении пика заболеваемости ниже 30%.
8. Интегрировать документацию с параметрами в формате Quarto в отчёт.

3 Теоретическое введение

Модель SIR, предложенная Кермаком и Маккендриком в 1927 году, описывает динамику эпидемии в популяции, разделённой на три группы: — ⊠ (Susceptible) — восприимчивые к заболеванию индивиды; — ⊠ (Infectious) — инфицированные, способные заражать восприимчивых; — ⊠ (Recovered) — выздоровевшие (или умершие).

Агентное моделирование позволяет преодолеть эти ограничения: — Каждый индивид моделируется отдельно с уникальными характеристиками. — Взаимодействия происходят локально в пространстве или социальной сети. — Процессы носят стохастический характер. — Можно учитывать гетерогенность контактов, мобильность, меры контроля.

Параметрами для нашей модели будут: — Ns — вектор численности населения по локациям; — β_und — вероятность передачи для невыявленных случаев; — β_det — вероятность передачи для выявленных случаев; — infection_period — длительность заболевания (дней); — detection_time — время до выявления заболевания; — death_rate — вероятность летального исхода; — reinfection_probability — вероятность повторного заражения; — migration_rates — матрица вероятностей миграции между локациями.

На каждом шаге (соответствует одному дню) для каждого агента выполняются следующие процессы : — Миграция — агент может переместиться в другую локацию с вероятностью, заданной матрицей migration_rates. — Передача инфекции — если агент инфицирован, он может заразить восприимчивых агентов в той же локации. Количество заражённых за день определяется пуассоновским

процессом с интенсивностью, зависящей от статуса выявления (β_und или β_det). — Обновление счётчика — для инфицированных увеличивается days_infected. — Выздоровление или смерть — если days_infected достиг infection_period, агент либо выздоравливает (переходит в R), либо умирает (удаляется из модели) с вероятностью death_rate. Выздоровевшие могут быть повторно инфицированы с вероятностью reinfection_probability.

4 Выполнение лабораторной работы

Начинаю выполнение лабораторной работы с создания новых файлов-скриптов. Начинаю с описания модели - sir_model.jl в папку scr, именно на основе него будут работать остальные скрипты (рис. 4.1).

Рисунок 4.1: Создание файла sir_model.jl

Далее пишу скрипт sir_run_basic.jl в scripts, как и затем все остальные скрипты тоже. Файл запускает один эксперимент с фиксированными параметрами и сохраняет динамику численности агентов. Это служит для проверки работоспособности модели и получения базового понимания эпидемического процесса (рис. 4.2) .

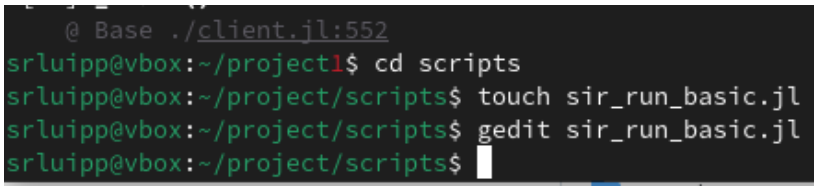


Рисунок 4.2: Создание скрипта sir_run_basic.jl

Создаю новый файл sir_scan_beta.jl - файл исследует, как изменение базовой заразности (β_und и пропорционально β_det) влияет на эпидемические

показатели: пик заболеваемости, долю переболевших, число умерших. Выполняется параметрическое сканирование с несколькими повторными прогонами для учёта стохастичности (рис. 4.3).

```
srluipp@vbox:~/project/scripts$ touch sir_scan_beta.jl
srluipp@vbox:~/project/scripts$ gedit sir_scan_beta.jl
```

Рисунок 4.3: Создание скрипта sir_scan_beta.jl

Заполняю код для файла sir_migration_effect.jl, который исследует, как интенсивность перемещения людей между городами влияет на скорость распространения эпидемии (время достижения пика) и масштаб пика. Инфекция начинается только в одном городе, остальные изначально здоровы (рис. 4.4).

```
srluipp@vbox:~/project/scripts$ touch sir_migration_effect.jl
srluipp@vbox:~/project/scripts$ gedit sir_migration_effect.jl
srluipp@vbox:~/project/scripts$
```

Рисунок 4.4: Создание sir_migration_effect.jl

Создаю скрипт sir_optimize_parameters.jl и sir_visualize_dynamics.jl. Первый ищет оптимальные комбинации параметров, минимизирующие одновременно два критерия: пиковую заболеваемость и долю умерших. Использует эволюционный алгоритм (Borg MOEA) из пакета BlackBoxOptim, второй - загружает результаты параметрического сканирования и строит единый составной график, объединяющий три панели: — пик эпидемии и конечная доля инфицированных; — число умерших; — доля выздоровевших(рис. 4.5).

```
srluipp@vbox:~/project/scripts$ touch sir_visualize_dynamics.jl
srluipp@vbox:~/project/scripts$ gedit sir_visualize_dynamics.jl
srluipp@vbox:~/project/scripts$
```

Рисунок 4.5: sir_optimize_parameters.jl

Для скрипта необходим пакет из Julia - Distributions (рис. 4.6).

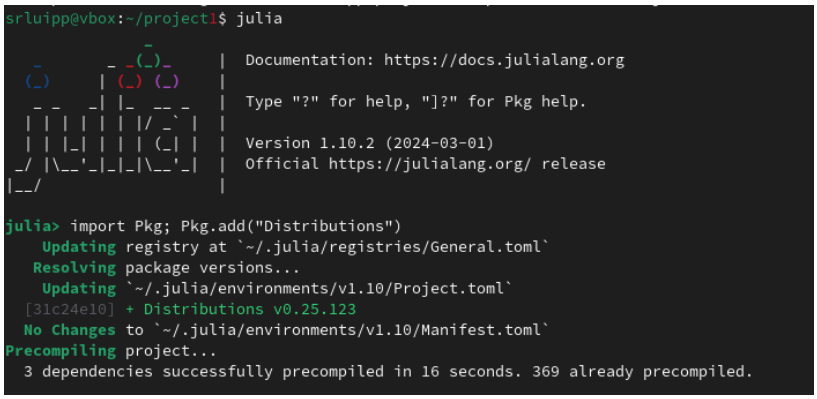


Рисунок 4.6: Distributions

Начинаю запускать через Julia получившиеся файлы: (рис. 4.7) (рис. 4.8).

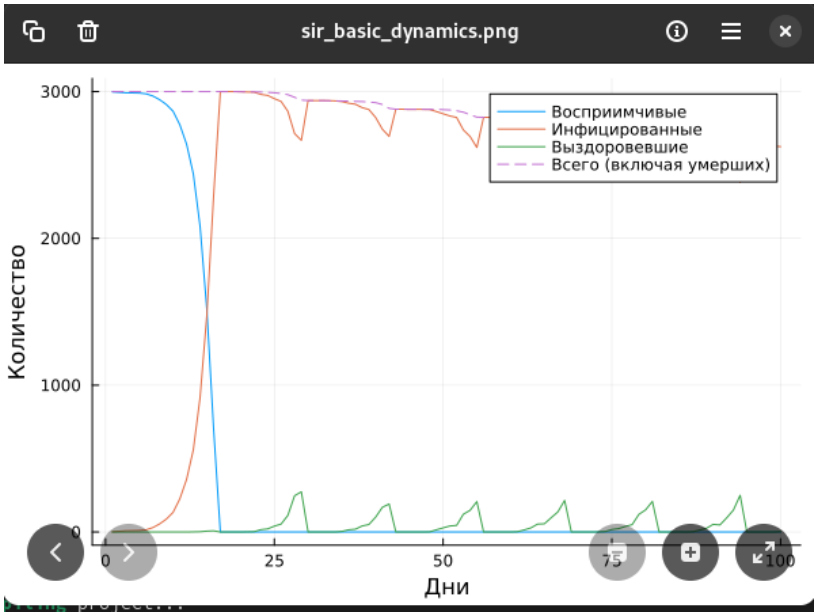


Рисунок 4.7: Статистика по количеству S, I, R

```
Завершён эксперимент с beta = 0.2, seed = 43
Завершён эксперимент с beta = 0.2, seed = 44
Завершён эксперимент с beta = 0.3, seed = 42
Завершён эксперимент с beta = 0.3, seed = 43
Завершён эксперимент с beta = 0.3, seed = 44
Завершён эксперимент с beta = 0.4, seed = 42
Завершён эксперимент с beta = 0.4, seed = 43
Завершён эксперимент с beta = 0.4, seed = 44
Завершён эксперимент с beta = 0.5, seed = 42
Завершён эксперимент с beta = 0.5, seed = 43
Завершён эксперимент с beta = 0.5, seed = 44
Завершён эксперимент с beta = 0.6, seed = 42
Завершён эксперимент с beta = 0.6, seed = 43
Завершён эксперимент с beta = 0.6, seed = 44
Завершён эксперимент с beta = 0.7, seed = 42
Завершён эксперимент с beta = 0.7, seed = 43
Завершён эксперимент с beta = 0.7, seed = 44
Завершён эксперимент с beta = 0.8, seed = 42
Завершён эксперимент с beta = 0.8, seed = 43
Завершён эксперимент с beta = 0.8, seed = 44
Завершён эксперимент с beta = 0.9, seed = 42
Завершён эксперимент с beta = 0.9, seed = 43
Завершён эксперимент с beta = 0.9, seed = 44
Завершён эксперимент с beta = 1.0, seed = 42
Завершён эксперимент с beta = 1.0, seed = 43
Завершён эксперимент с beta = 1.0, seed = 44
Результаты сохранены в data/beta_scan_all.csv и plots/beta_scan.png
```

Рисунок 4.8: Запуск эксперимента

(рис. 4.9).

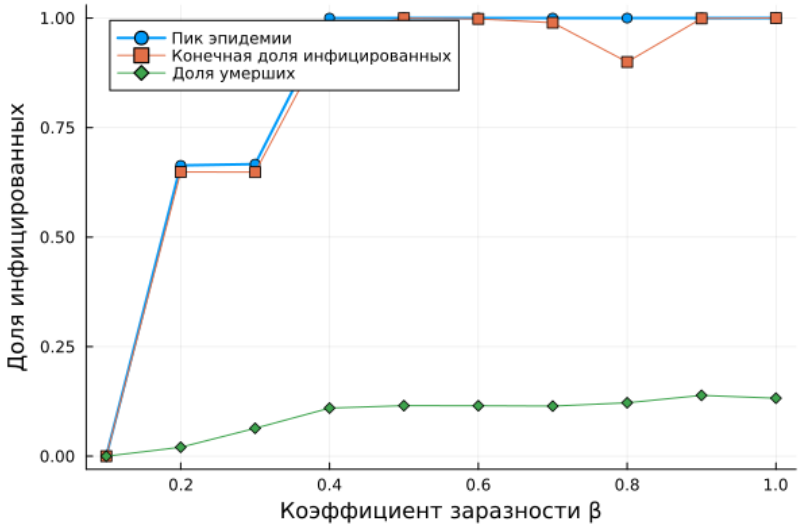


Рисунок 4.9: Доля инфицированных от количества заразности

Проводятся эксперименты с миграцией (рис. 4.10) (рис. 4.11).

```
srluipp@vbox:~/project$ julia --project=. scripts/sir_migration_effect.jl
Завершён эксперимент с migration_intensity =
0.0, seed =
42
Завершён эксперимент с migration_intensity =
0.0, seed =
43
Завершён эксперимент с migration_intensity =
0.0, seed =
44
Завершён эксперимент с migration_intensity =
0.1, seed =
42
Завершён эксперимент с migration_intensity =
0.1, seed =
43
Завершён эксперимент с migration_intensity =
0.1, seed =
44
```

Рисунок 4.10: Эксперименты с миграцией

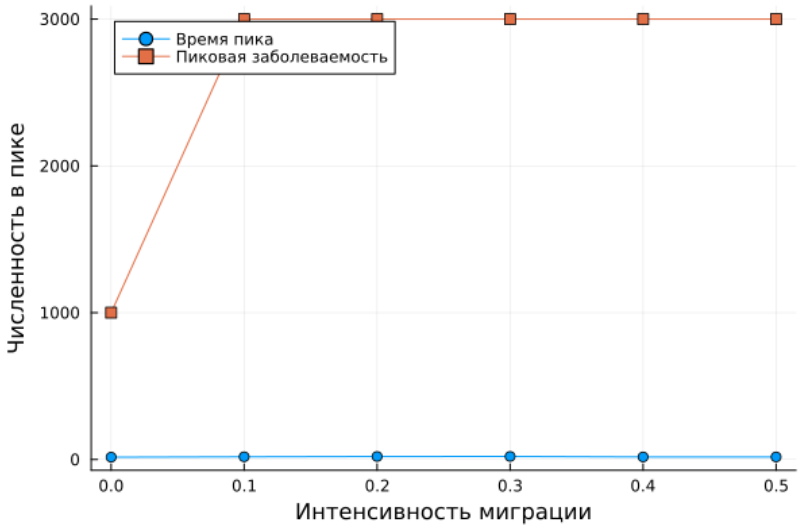
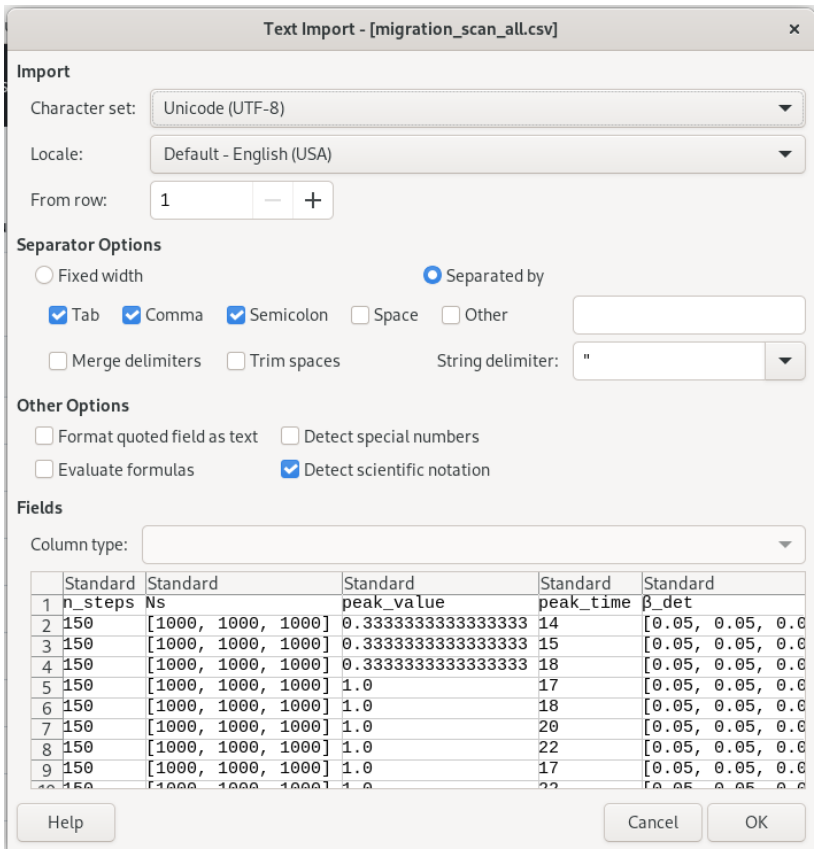


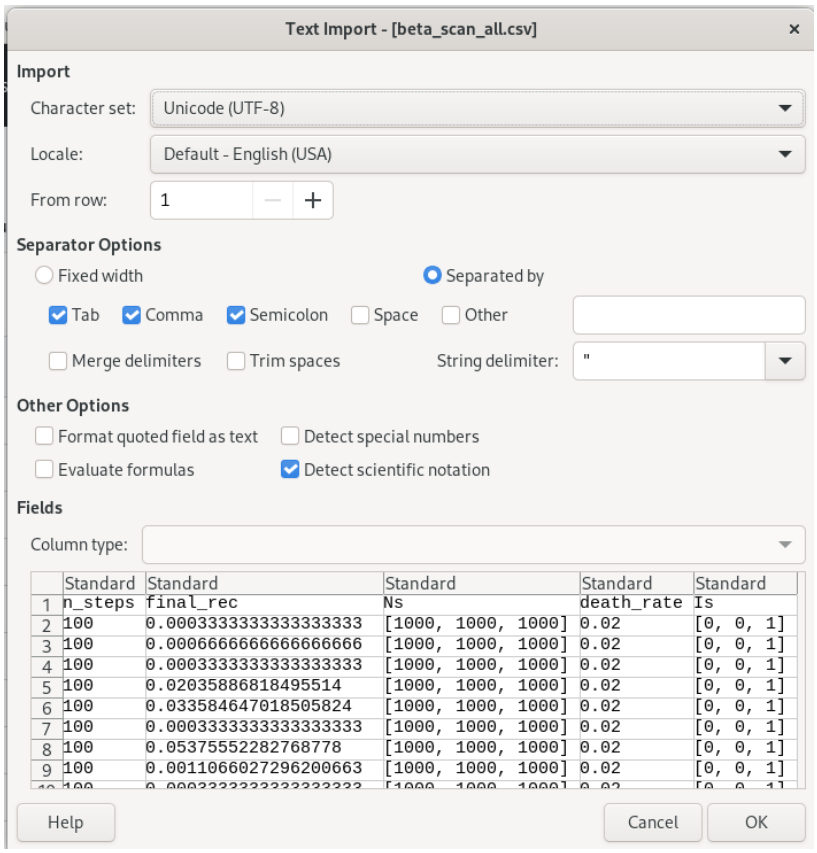
Рисунок 4.11: Численность людей в пике от интенсивности миграции

В формате cvs сохранились результаты экспериментов файлов с миграцией и числом бета (рис. 4.12) (рис. 4.13).



	Standard	Standard	Standard	Standard	Standard
1	n_steps	Ns	peak_value	peak_time	β_det
2	150	[1000, 1000, 1000]	0.3333333333333333	14	[0.05, 0.05, 0.0
3	150	[1000, 1000, 1000]	0.3333333333333333	15	[0.05, 0.05, 0.0
4	150	[1000, 1000, 1000]	0.3333333333333333	18	[0.05, 0.05, 0.0
5	150	[1000, 1000, 1000]	1.0	17	[0.05, 0.05, 0.0
6	150	[1000, 1000, 1000]	1.0	18	[0.05, 0.05, 0.0
7	150	[1000, 1000, 1000]	1.0	20	[0.05, 0.05, 0.0
8	150	[1000, 1000, 1000]	1.0	22	[0.05, 0.05, 0.0
9	150	[1000, 1000, 1000]	1.0	17	[0.05, 0.05, 0.0

Рисунок 4.12: migration_scan_all.cvs



	Standard	Standard	Standard	Standard	Standard
1	n_steps	final_rec	Ns	death_rate	Is
2	100	0.0003333333333333333	[1000, 1000, 1000]	0.02	[0, 0, 1]
3	100	0.0006666666666666666	[1000, 1000, 1000]	0.02	[0, 0, 1]
4	100	0.0003333333333333333	[1000, 1000, 1000]	0.02	[0, 0, 1]
5	100	0.02035886818495514	[1000, 1000, 1000]	0.02	[0, 0, 1]
6	100	0.033584647018505824	[1000, 1000, 1000]	0.02	[0, 0, 1]
7	100	0.0003333333333333333	[1000, 1000, 1000]	0.02	[0, 0, 1]
8	100	0.05375552282768778	[1000, 1000, 1000]	0.02	[0, 0, 1]
9	100	0.0011066027296200663	[1000, 1000, 1000]	0.02	[0, 0, 1]

Рисунок 4.13: beta_scan_all.cvs

Для сводного графика sir_visualize_dynamics.jl устанавливаем нужный пакет из Julia (рис. 4.14).

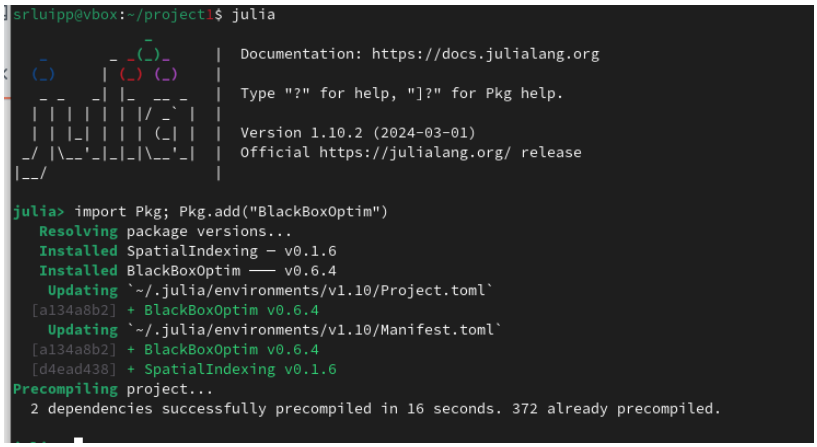


Рисунок 4.14: BlackBoxOptim

На выходе получаем некоторые найденные параметры в консоль и общий сводный график (рис. 4.15) (рис. 4.16).

```
64, true, typeof(sum)}, BlackBoxOptim.ProblemEvaluator{Tuple{Float64, Float64}, IndexedTupleFitne
ss{2, Float64}, EpsBoxArchive{2, Float64, EpsBoxDominanceFitnessScheme{2, Float64, true, typeof(s
um)}}, FunctionBasedProblem{typeof(cost_multi), ParetoFitnessScheme{2, Float64, true, typeof(sum)
}, ContinuousRectSearchSpace, Nothing}}, FitPopulation{IndexedTupleFitness{2, Float64}}, FixedGen
eticOperatorsMixture, RandomBound{ContinuousRectSearchSpace}}
0.00 secs, 0 evals, 0 steps
pop.size=50 arch.size=0 n.restarts=0

Optimization stopped after 1 steps and 1534.16 seconds
Termination reason: Max time (120.0 s) reached
Steps per second = 0.00
Function evals per second = 0.03
Improvements/step = NaN
Total function evaluations = 51


Best candidate found: [0.112984, 5.82966, 0.0550191]

Fitness: (0.00067, 0.00000) agg=0.00067

Оптимальные параметры:
β_und = 0.11298373996811871
Время выявления = 6 дней
Смертность = 0.05501911495696024
Достигнутые показатели:
Пик заболеваемости: 0.0006666666666666666
Доля умерших: 0.0
```

Рисунок 4.15: Вывод файла sir_visualize_dynamics.jl в консоль

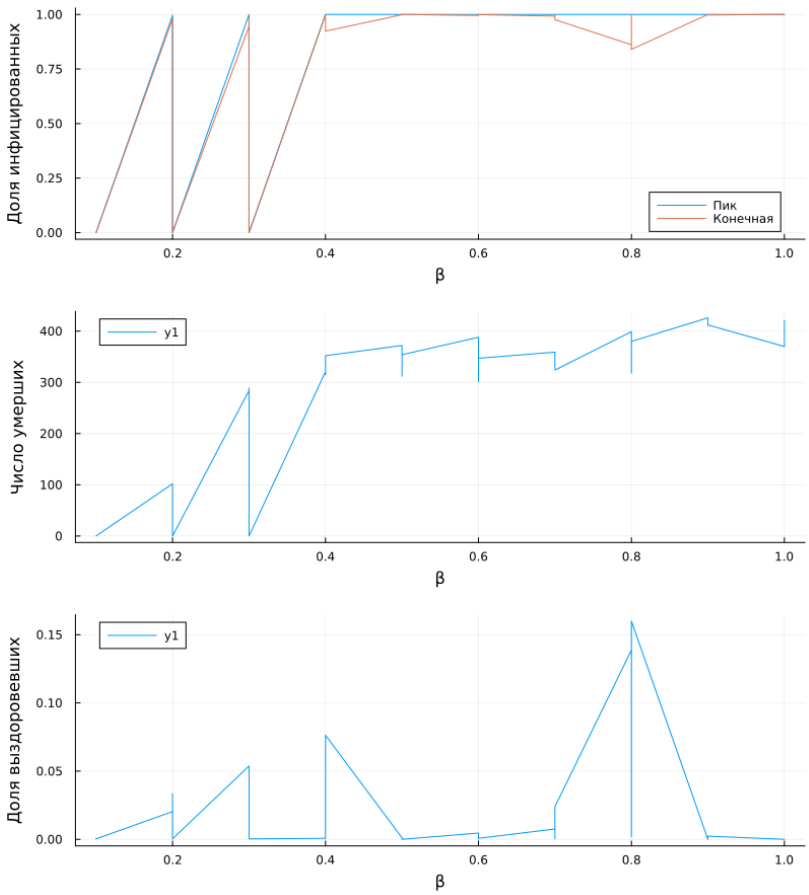


Рисунок 4.16: Итоговый отчет comprehensive_analysis.png

Далее выполняю домашную работу - создаю новый файл в котором опишу все здания - homework.jl. Получаю графики в plots и некоторые найденные результаты в консоль (рис. 4.17) (рис. 4.18) (рис. 4.19) (рис. 4.20) (рис. 4.21) (рис. 4.22).

```
 задание 1
R0 = 4.2

 задание 2
Минимальное β для эпидемии: 0.17
Теоретический порог R0=1 соответствует β = 0.07142857142857142

 задание 3

 задание 4
0.0 → 9
0.1 → 12
0.2 → 12
0.3 → 12
0.4 → 12
0.5 → 12

 задание 5
Запуск без карантина...
Запуск с карантином (порог=10%)...
Без карантина: пик = 3000.0 человек
С карантином: пик = 3000.0 человек

 задание 6
Starting optimization with optimizer DiffEvoOpt{FitPopulation{Float64}, RadiusLimitedSelector, Bl
```

Рисунок 4.17: Полученные выводы в консоль для заданий 1-5

```
 задание 6
Starting optimization with optimizer DiffEvoOpt{FitPopulation{Float64}, RadiusLimitedSel
ackBoxOptim.AdaptiveDiffEvoRandBin{3}, RandomBound{ContinuousRectSearchSpace}}
0.00 secs, 0 evals, 0 steps
27.20 secs, 2 evals, 1 steps, fitness=10.216333333

Optimization stopped after 2 steps and 65.93 seconds
Termination reason: Max time (60.0 s) reached
Steps per second = 0.03
Function evals per second = 0.06
Improvements/step = NaN
Total function evaluations = 4


Best candidate found: [0.322338, 9.38969, 0.023474]

Fitness: 10.129222222

Оптимум: [0.32233771817719314, 9.389687522809803, 0.023474024844178442]
srluipp@vbox:~/project$
```

Рисунок 4.18: Вывод в консоль задания 6

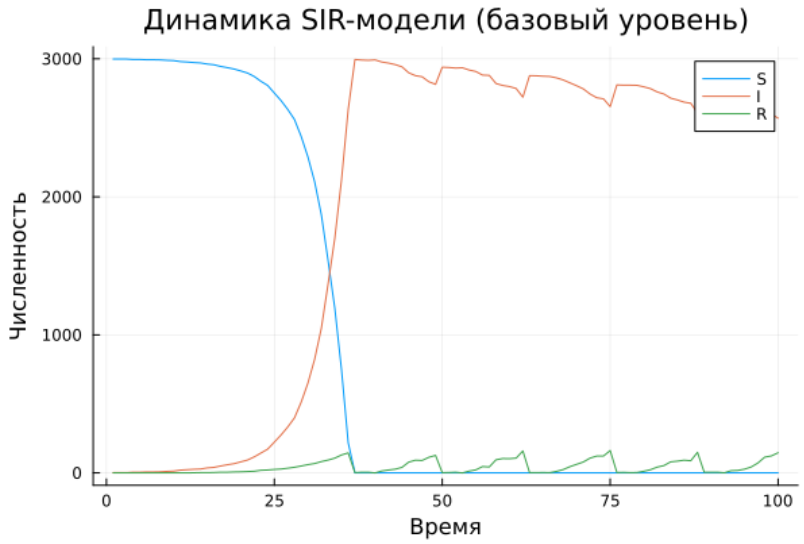


Рисунок 4.19: Получившийся график задания 1

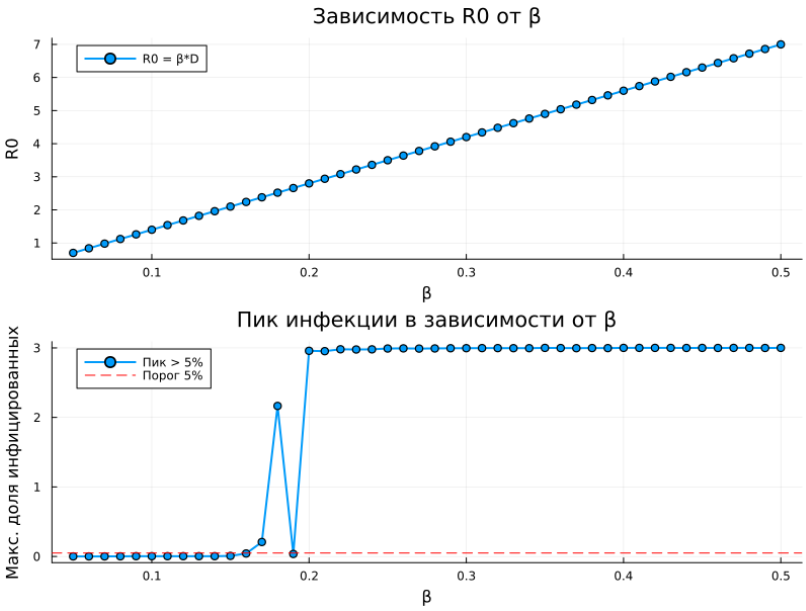


Рисунок 4.20: График задания 3

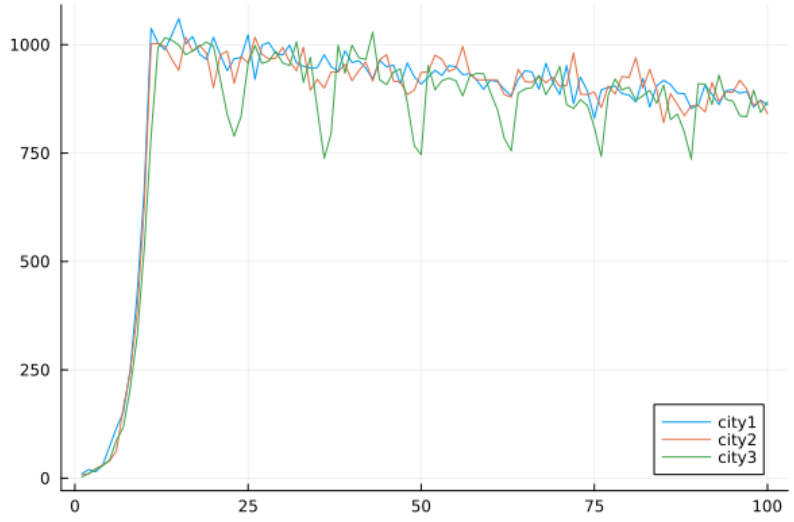


Рисунок 4.21: График задания 4

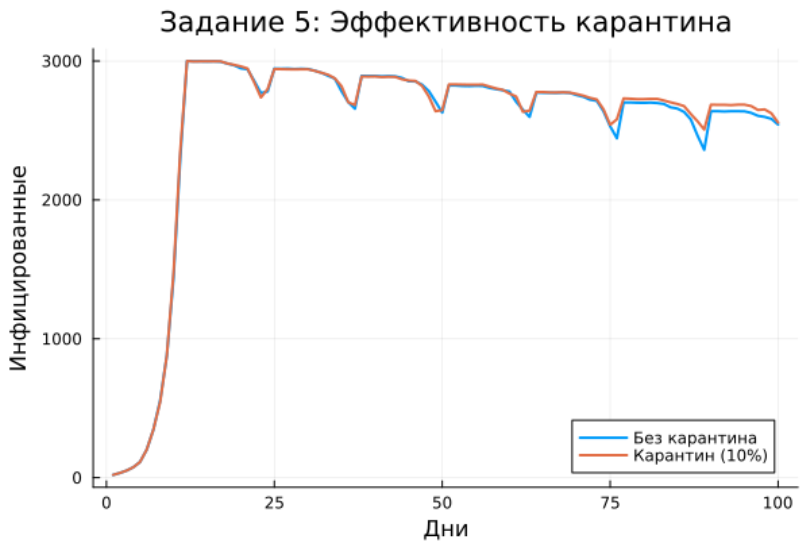


Рисунок 4.22: График задания 5

```
using DrWatson
@quickactivate "project"
using Agents, DataFrames, Plots
using JLD2
include("/home/srluipp/project/src/sir_model.jl")
```

```
total_count (generic function with 1 method)
```

Параметры эксперимента

```julia
params = Dict(
:Ns => [1000, 1000, 1000],
:β_und => [0.5, 0.5, 0.5],
:β_det => [0.05, 0.05, 0.05],
:infection_period => 14,
:detection_time => 7,
:death_rate => 0.02,
:reinfection_probability => 0.1,
:Is => [0, 0, 1],
:seed => 42,
:n_steps => 100,
)
```

```
Dict{Symbol, Any} with 10 entries:
  :Is                      => [0, 0, 1]
  :seed                    => 42
  :infection_period        => 14
  :n_steps                 => 100
  :β_und                   => [0.5, 0.5, 0.5]
  :Ns                      => [1000, 1000, 1000]
  :detection_time          => 7
  :reinfection_probability => 0.1
  :β_det                   => [0.05, 0.05, 0.05]
  :death_rate              => 0.02
```

Инициализация модели

```julia
model = initialize_sir(; params...)
```

```
StandardABM with 3000 agents of type Person
 agents container: Dict
 space: GraphSpace with 3 positions and 3 edges
 scheduler: fastest
 properties: C, infection_period, β_und, Ns, migration_rates, detection_
```

Подготовка массивов для хранения данных

```julia
times = Int[]
S_vals = Int[]
I_vals = Int[]
R_vals = Int[]
total_vals = Int[]
```

```
Int64[]
```

Запуск симуляции вручную

```julia
for step = 1:params[:n_steps]
Agents.step!(model, 1)
push!(times, step)
push!(S_vals, susceptible_count(model))
push!(I_vals, infected_count(model))
push!(R_vals, recovered_count(model))
push!(total_vals, total_count(model))
end
```

Создаём DataFrame для удобства (опционально)

```
agent_df =
DataFrame(time = times, susceptible = S_vals, infected = I_vals, recover
model_df = DataFrame(time = times, total = total_vals)
```

	time	total
	Int64	Int64
1	1	3000
2	2	3000
3	3	3000
4	4	3000
5	5	3000
6	6	3000
7	7	3000
8	8	3000
9	9	3000
10	10	3000
11	11	3000
12	12	3000
13	13	3000
14	14	3000
15	15	3000
16	16	3000
17	17	3000
18	18	3000
19	19	3000
20	20	2999
21	21	2998
22	22	2998
23	23	2997
24	24	2995
25	25	2992
26	26	2987
27	27	2979
28	28	2961
29	29	2942
30	30	2939
…	…	…

Визуализация

```
plot(
agent_df.time,
agent_df.susceptible,
label = "Восприимчивые",
xlabel = "Дни",
ylabel = "Количество",
)
plot!(agent_df.time, agent_df.infected, label = "Инфицированные")
plot!(agent_df.time, agent_df.recovered, label = "Выздоровевшие")
plot!(agent_df.time, model_df.total, label = "Всего (включая умерших)",
savefig(plotsdir("sir_basic_dynamics.png"))
```

```
"/home/srluipp/plots/sir_basic_dynamics.png"
```

Сохранение данных

```
@save datadir("sir_basic_agent.jld2") agent_df
@save datadir("sir_basic_model.jld2") model_df
```

4.1 Сканирование коэффициента заразности β

4.1.1 Описание

Данный скрипт исследует, как изменение базовой заразности (β) влияет на эпидемические показатели: пик заболеваемости, долю переболевших и число умерших. Выполняется параметрическое сканирование с несколькими повторными прогонами для учёта стохастичности.

```julia
using DrWatson
@quickactivate "project"
using Agents, DataFrames, Plots, CSV, Random, Statistics

include("/home/srluipp/project/src/sir_model.jl")
```

```
total_count (generic function with 1 method)
```

4.1.2 Функция для запуска одного эксперимента и возврата метрик

Функция принимает словарь параметров `p` и возвращает: - `peak`: пиковая доля инфицированных - `final_inf`: конечная доля инфицированных - `final_rec`: конечная доля выздоровевших - `deaths`: общее число умерших

```julia
function run_experiment(p)
    beta = p[:beta]
    β_und = fill(beta, 3)
    β_det = fill(beta/10, 3)

    model = initialize_sir(;
        Ns = p[:Ns],
        β_und = β_und,
        β_det = β_det,
        infection_period = p[:infection_period],
        detection_time = p[:detection_time],
        death_rate = p[:death_rate],
        reinfection_probability = p[:reinfection_probability],
        Is = p[:Is],
```

```
        seed = p[:seed],
    )

    infected_fraction(model) = count(a.status == :I for a in allagents(m
    peak_infected = 0.0

    for step in 1:p[:n_steps]
        Agents.step!(model, 1)
        frac = infected_fraction(model)
        if frac > peak_infected
            peak_infected = frac
        end
    end

    final_infected = infected_fraction(model)
    final_recovered = count(a.status == :R for a in allagents(model)) /
    total_deaths = sum(p[:Ns]) - nagents(model)

    return (
        peak = peak_infected,
        final_inf = final_infected,
        final_rec = final_recovered,
        deaths = total_deaths,
    )
end
```

```
run_experiment (generic function with 1 method)
```

4.1.3 Параметры сканирования

- Диапазон β: от 0.1 до 1.0 с шагом 0.1
- Количество повторных прогонов для каждого β: 3 (разные seed)

```
beta_range = 0.1:0.1:1.0
seeds = [42, 43, 44]
```

```
3-element Vector{Int64}:
 42
 43
 44
```

4.1.4 Формирование списка параметров

```
params_list = []
for b in beta_range
    for s in seeds
        push!(params_list, Dict(
            :beta => b,
            :Ns => [1000, 1000, 1000],
            :infection_period => 14,
            :detection_time => 7,
            :death_rate => 0.02,
            :reinfection_probability => 0.1,
            :Is => [0, 0, 1],
            :seed => s,
            :n_steps => 100,
        ))
```

```
    end
end
```

4.1.5 Запуск экспериментов

```
results = []
for params in params_list
    data = run_experiment(params)
    push!(results, merge(params, Dict(pairs(data))))
    println("Завершён эксперимент с beta = $(params[:beta]), seed = $(pa
end
```

```
Завершён эксперимент с beta = 0.1, seed = 42
Завершён эксперимент с beta = 0.1, seed = 43
Завершён эксперимент с beta = 0.1, seed = 44
Завершён эксперимент с beta = 0.2, seed = 42
Завершён эксперимент с beta = 0.2, seed = 43
Завершён эксперимент с beta = 0.2, seed = 44
Завершён эксперимент с beta = 0.3, seed = 42
Завершён эксперимент с beta = 0.3, seed = 43
Завершён эксперимент с beta = 0.3, seed = 44
Завершён эксперимент с beta = 0.4, seed = 42
Завершён эксперимент с beta = 0.4, seed = 43
Завершён эксперимент с beta = 0.4, seed = 44
Завершён эксперимент с beta = 0.5, seed = 42
Завершён эксперимент с beta = 0.5, seed = 43
Завершён эксперимент с beta = 0.5, seed = 44
Завершён эксперимент с beta = 0.6, seed = 42
Завершён эксперимент с beta = 0.6, seed = 43
```

```
Завершён эксперимент с beta = 0.6, seed = 44
Завершён эксперимент с beta = 0.7, seed = 42
Завершён эксперимент с beta = 0.7, seed = 43
Завершён эксперимент с beta = 0.7, seed = 44
Завершён эксперимент с beta = 0.8, seed = 42
Завершён эксперимент с beta = 0.8, seed = 43
Завершён эксперимент с beta = 0.8, seed = 44
Завершён эксперимент с beta = 0.9, seed = 42
Завершён эксперимент с beta = 0.9, seed = 43
Завершён эксперимент с beta = 0.9, seed = 44
Завершён эксперимент с beta = 1.0, seed = 42
Завершён эксперимент с beta = 1.0, seed = 43
Завершён эксперимент с beta = 1.0, seed = 44
```

4.1.6 Сохранение результатов

```julia
df = DataFrame(results)
CSV.write(datadir("beta_scan_all.csv"), df)
```

```
"/home/srluipp/data/beta_scan_all.csv"
```

4.1.7 Усреднение по повторным прогонам

```julia
grouped = combine(groupby(df, [:beta]),
    :peak => mean => :mean_peak,
    :final_inf => mean => :mean_final_inf,
    :deaths => mean => :mean_deaths,
)
```

	beta	mean_peak	mean_final_inf	mean_deaths
	Float64	Float64	Float64	Float64
1	0.1	0.000444444	0.0	0.0
2	0.2	0.663745	0.648685	61.3333
3	0.3	0.666666	0.648379	191.0
4	0.4	1.0	0.965633	329.333
5	0.5	1.0	0.999619	346.0
6	0.6	1.0	0.998094	345.333
7	0.7	1.0	0.989378	343.0
8	0.8	1.0	0.899806	365.667
9	0.9	1.0	0.999227	416.333
10	1.0	1.0	1.0	397.0

4.1.8 Визуализация

```
plot(grouped.beta, grouped.mean_peak,
     label = "Пик эпидемии",
     xlabel = "Коэффициент заразности β",
     ylabel = "Доля инфицированных",
     marker = :circle,
     linewidth = 2)
plot!(grouped.beta, grouped.mean_final_inf,
      label = "Конечная доля инфицированных",
      marker = :square)
plot!(grouped.beta, grouped.mean_deaths ./ 3000,
      label = "Доля умерших",
      marker = :diamond)
savefig(plotsdir("beta_scan.png"))
```

```
println("Результаты сохранены в data/beta_scan_all.csv и plots/beta_scan
```

```
Результаты сохранены в data/beta_scan_all.csv и plots/beta_scan.png
```

4.1.9 Выводы

1. Сканирование коэффициента заразности β позволяет выявить пороговое значение, при котором возникает эпидемия. Теоретический порог соответствует $R_0 = 1$, т.е. $\beta = \gamma$.

2. С ростом β наблюдается нелинейное увеличение пиковой доли инфицированных: чем выше заразность, тем быстрее распространяется инфекция и тем больше людей одновременно болеют.

3. Конечная доля инфицированных (переболевших) также возрастает с увеличением β, стремясь к насыщению, что связано с формированием популяционного иммунитета.

4. Доля умерших коррелирует с пиком заболеваемости: более интенсивная эпидемия приводит к большему числу летальных исходов.

5. Наличие повторных прогонов с разными seed позволяет оценить стохастическую изменчивость результатов, что особенно важно при малых значениях β, где случайные флуктуации могут существенно влиять на исход.

6. Полученные данные сохраняются в CSV-файл для дальнейшего анализа, а график наглядно демонстрирует зависимость ключевых показателей от заразности инфекции.

```
using DrWatson
@quickactivate "project"
```

```
using BlackBoxOptim, Random, Statistics
include("/home/srluipp/project/src/sir_model.jl")
```

```
total_count (generic function with 1 method)
```

Целевая функция: минимизируем пиковую заболеваемость и смертность

```
function cost_multi(x)
model = initialize_sir(;
Ns = [1000, 1000, 1000],
β_und = fill(x[1], 3),
β_det = fill(x[1]/10, 3),
infection_period = 14,
detection_time = round(Int, x[2]),
death_rate = x[3],
reinfection_probability = 0.1,
Is = [0, 0, 1],
seed = 42,
n_steps = 100,
)
infected_frac(model) = count(a.status == :I for a in allagents(model)) /
dead_count(model) = 3000 - nagents(model)
peak_infected = 0.0
replicates = 5
peak_vals = Float64[]
dead_vals = Int[]
for rep = 1:replicates
model = initialize_sir(;
Ns = [1000, 1000, 1000],
```

```
β_und = fill(x[1], 3),
β_det = fill(x[1]/10, 3),
infection_period = 14,
detection_time = round(Int, x[2]),
death_rate = x[3],
reinfection_probability = 0.1,
Is = [0, 0, 1],
seed = 42 + rep,
n_steps = 100,
)
for step = 1:100
Agents.step!(model, 1)
frac = infected_frac(model)
if frac > peak_infected
peak_infected = frac
end
end
push!(peak_vals, peak_infected)
push!(dead_vals, dead_count(model))
end
return (mean(peak_vals), mean(dead_vals) / 3000) # доля умерших
end
result = bboptimize(
cost_multi,
Method = :borg_moea,
FitnessScheme = ParetoFitnessScheme{2}(is_minimizing = true),
SearchRange = [
(0.1, 1.0), # β_und
```

```
(3.0, 14.0), # detection_time
(0.01, 0.1), # death_rate
],
NumDimensions = 3,
MaxTime = 120, # 2 минуты
TraceMode = :compact,
)
best = best_candidate(result)
fitness = best_fitness(result)
println("Оптимальные параметры:")
println("β_und = $(best[1])")
println("Время выявления = $(round(Int, best[2])) дней")
println("Смертность = $(best[3])")
println("Достигнутые показатели:")
println("Пик заболеваемости: $(fitness[1])")
println("Доля умерших: $(fitness[2])")
save(datadir("optimization_result.jld2"), Dict("best" ⇒ best, "fitness"
```

```
Starting optimization with optimizer BlackBoxOptim.BorgMOEA{EpsBoxDomina
0.00 secs, 0 evals, 0 steps
pop.size=50 arch.size=0 n.restarts=0

Optimization stopped after 1 steps and 1387.01 seconds
Termination reason: Max time (120.0 s) reached
Steps per second = 0.00
Function evals per second = 0.04
Improvements/step = NaN
Total function evaluations = 51
```

```
Best candidate found: [0.120378, 5.45868, 0.0938334]

Fitness: (0.00067, 0.00007) agg=0.00073

Оптимальные параметры:
β_und = 0.12037815041415895
Время выявления = 5 дней
Смертность = 0.09383340427152549
Достигнутые показатели:
Пик заболеваемости: 0.0006666666666666666
Доля умерших: 6.666666666666667e-5
```

```julia
using DrWatson
@quickactivate "project"
using Agents, DataFrames, Plots, CSV
include("/home/srluipp/project/src/sir_model.jl")
```

```
total_count (generic function with 1 method)
```

Загружаем результаты сканирования

```julia
df = CSV.read(datadir("beta_scan_all.csv"), DataFrame)
```

	n_steps	final_rec	Ns	death_rate	Is	beta	infection_period
	Int64	Float64	String31	Float64	String15	Float64	Int64
1	100	0.000333333	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.1	14
2	100	0.000666667	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.1	14
3	100	0.000333333	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.1	14
4	100	0.0203589	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.2	14
5	100	0.0335846	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.2	14
6	100	0.000333333	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.2	14
7	100	0.0537555	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.3	14
8	100	0.0011066	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.3	14
9	100	0.000333333	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.3	14
10	100	0.000746547	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.4	14
11	100	0.0260708	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.4	14
12	100	0.076284	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.4	14
13	100	0.00114155	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.5	14
14	100	0.0	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.5	14
15	100	0.0	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.5	14
16	100	0.00459418	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.6	14
17	100	0.000370508	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.6	14
18	100	0.000753864	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.6	14
19	100	0.00757289	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.7	14
20	100	0.00037679	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.7	14
21	100	0.0239163	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.7	14
22	100	0.138793	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.8	14
23	100	0.00186428	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.8	14
24	100	0.159924	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.8	14
25	100	0.0	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.9	14
26	100	0.0	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.9	14
27	100	0.00231839	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.9	14
28	100	0.0	[1000, 1000, 1000]	0.02	[0, 0, 1]	1.0	14
29	100	0.0	[1000, 1000, 1000]	0.02	[0, 0, 1]	1.0	14
30	100	0.0	[1000, 1000, 1000]	0.02	[0, 0, 1]	1.0	14

Создаём составной график

```
p1 = plot(df.beta, df.peak, label = "Пик", xlabel = "β", ylabel = "Доля
plot!(p1, df.beta, df.final_inf, label = "Конечная")
p2 = plot(df.beta, df.deaths, xlabel = "β", ylabel = "Число умерших")
p3 = plot(df.beta, df.final_rec, xlabel = "β", ylabel = "Доля выздоровев
plot(p1, p2, p3, layout = (3, 1), size = (800, 900))
savefig(plotsdir("comprehensive_analysis.png"))
```

```
"/home/srluipp/plots/comprehensive_analysis.png"
```

```
using DrWatson
@quickactivate "project"
using Agents, Plots, Statistics, Random, BlackBoxOptim, JLD2, DataFrames
include("/home/srluipp/project/src/sir_model.jl")
```

```
total_count (generic function with 1 method)
```

задание 1

```
println("\nЗадание 1")
model1 = initialize_sir(;
    Ns = [1000, 1000, 1000],
    β_und = [0.3, 0.3, 0.3],
    β_det = [0.03, 0.03, 0.03],
    infection_period = 14,
    detection_time = 7,
    death_rate = 0.02,
    reinfection_probability = 0.1,
    Is = [0, 0, 1],
```

```
    seed = 42
)
times = 1:100
S_vals = Float64[]
I_vals = Float64[]
R_vals = Float64[]
for _ in times
    Agents.step!(model1, 1)
    push!(S_vals, susceptible_count(model1))
    push!(I_vals, infected_count(model1))
    push!(R_vals, recovered_count(model1))
end
plot(times, S_vals, label="S")
plot!(times, I_vals, label="I")
plot!(times, R_vals, label="R")
xlabel!("Время")
ylabel!("Численность")
title!("Динамика SIR-модели (базовый уровень)")
savefig(plotsdir("task1.png"))
```

```
Задание 1

"/home/srluipp/plots/task1.png"
```

Вычисление R_0

```
γ = 1 / 14
R₀ = 0.3 / γ
```

```
println("R₀ = ", R₀)
println("Наблюдаемая динамика: I сначала растёт, затем уменьшается, что
```

```
R₀ = 4.2
Наблюдаемая динамика: I сначала растёт, затем уменьшается, что соответст
```

задание 2

```
println("\n задание 2")
function find_epidemic_threshold(β_range=0.05:0.01:0.5; n_steps=100, N=1
    results = []
    for β in β_range
        model = initialize_sir(;
            Ns = [N, N, N],
            β_und = [β, β, β],
            β_det = [β/10, β/10, β/10],
            infection_period = 14,
            detection_time = 7,
            death_rate = 0.02,
            reinfection_probability = 0.1,
            Is = [1,0,0],
            seed = seed
        )
        max_I = 0.0
        for _ in 1:n_steps
            Agents.step!(model,1)
            current_I = infected_count(model)
            max_I = max(max_I, current_I)
        end
```

```
        epidemic = max_I > 0.05*N
        R0 = β*14
        push!(results, Dict(
            :β => β,
            :R0 => R0,
            :epidemic => epidemic,
            :max_I => max_I,
            :max_I_N => max_I/N
        ))
    end
    df = DataFrame(results)
    CSV.write(datadir("task2_threshold.csv"), df)
    p1 = plot(df.β, df.R0,
             xlabel="β", ylabel="R0",
             title="Зависимость R0 от β",
             label="R0 = β*D",
             lw=2, marker=:circle)
    p2 = plot(df.β, df.max_I./N,
             xlabel="β", ylabel="Макс. доля инфицированных",
             title="Пик инфекции в зависимости от β",
             label="Пик > 5%",
             lw=2, marker=:circle)
    hline!(p2, [0.05], label="Порог 5%", color=:red, ls=:dash)
    combined = plot(p1, p2, layout=(2,1), size=(800,600))
    savefig(combined, plotsdir("task2_threshold.png"))
    epidemic_cases = filter(:epidemic => ==(true), df)
    min_β = epidemic_cases.β[1]
    println("Минимальное β для эпидемии: ", min_β)
```

```
    println("Теоретический порог R0=1 соответствует β = ", 1/14)
    return df
end
task2_results = find_epidemic_threshold()
```

```
 задание 2
Минимальное β для эпидемии: 0.17
Теоретический порог R0=1 соответствует β = 0.07142857142857142
```

	R0	epidemic	max_I	max_I_N	β
	Float64	Bool	Float64	Float64	Float64
1	0.7	0	1.0	0.001	0.05
2	0.84	0	1.0	0.001	0.06
3	0.98	0	1.0	0.001	0.07
4	1.12	0	2.0	0.002	0.08
5	1.26	0	4.0	0.004	0.09
6	1.4	0	4.0	0.004	0.1
7	1.54	0	4.0	0.004	0.11
8	1.68	0	4.0	0.004	0.12
9	1.82	0	4.0	0.004	0.13
10	1.96	0	4.0	0.004	0.14
11	2.1	0	8.0	0.008	0.15
12	2.24	0	43.0	0.043	0.16
13	2.38	1	208.0	0.208	0.17
14	2.52	1	2165.0	2.165	0.18
15	2.66	0	36.0	0.036	0.19
16	2.8	1	2958.0	2.958	0.2
17	2.94	1	2953.0	2.953	0.21
18	3.08	1	2980.0	2.98	0.22
19	3.22	1	2977.0	2.977	0.23
20	3.36	1	2979.0	2.979	0.24
21	3.5	1	2991.0	2.991	0.25
22	3.64	1	2993.0	2.993	0.26
23	3.78	1	2990.0	2.99	0.27
24	3.92	1	2992.0	2.992	0.28
25	4.06	1	2995.0	2.995	0.29
26	4.2	1	2996.0	2.996	0.3
27	4.34	1	2997.0	2.997	0.31
28	4.48	1	2997.0	2.997	0.32
29	4.62	1	2996.0	2.996	0.33
30	4.76	1	2996.0	2.996	0.34
…	…	…	…	…	…

задание 3

```
println("\n задание 3")
```

```
задание 3
```

Инициализация модели с 3 городами

```
model3 = initialize_sir(;
    Ns = [1000, 1000, 1000],
    β_und = [0.7, 0.5, 0.3],
    β_det = [0.07, 0.05, 0.03],
    Is = [10, 0, 0],
    seed = 42
)
```

```
StandardABM with 3000 agents of type Person
 agents container: Dict
 space: GraphSpace with 3 positions and 3 edges
 scheduler: fastest
 properties: C, infection_period, β_und, Ns, migration_rates, detection_
```

Симуляция и сбор данных

```
I_city = [Float64[] for _ in 1:3]
for _ in 1:100
    Agents.step!(model3, 1)
    for c in 1:3
        push!(I_city[c], count(a -> a.pos == c && a.status == :I, allage
    end
end
```

Визуализация

```
p3 = plot(I_city,
     label = ["city1" "city2" "city3"],
     xlabel = "День",
     ylabel = "Число инфицированных",
     title = "Динамика инфекции в городах",
     lw = 2)
savefig(p3, plotsdir("task3.png"))


p3 = plot(I_city[1], label="city1")
plot!(p3, I_city[2], label="city2")
plot!(p3, I_city[3], label="city3")
savefig(p3, plotsdir("task3.png"))
```

```
"/home/srluipp/plots/task3.png"
```

задание 4

```
println("\n задание 4")
function create_migration_matrix(C,x)
    M = ones(C,C).*x/(C-1)
    for i in 1:C
        M[i,i] = 1-x
    end
    return M
end

function peak_time(x)
```

```
    model = initialize_sir(;
        Ns=[1000,1000,1000],
        β_und=[0.5,0.5,0.5],
        β_det=[0.05,0.05,0.05],
        migration_rates=create_migration_matrix(3,x),
        Is=[10,0,0],
        seed=42
    )

    peak_t = 0
    max_i = 0

    for t in 1:100
        Agents.step!(model,1)
        i = infected_count(model)
        if i > max_i
            max_i = i
            peak_t = t
        end
    end
    return peak_t
end

for x in 0.0:0.1:0.5
    println(x, " → ", peak_time(x))
end
```

задание 4

```
0.0 → 9
0.1 → 12
0.2 → 12
0.3 → 12
0.4 → 12
0.5 → 12
```

здание 5

```julia
println("\n задание 5")
function run_with_quarantine(use_quarantine, threshold)
    if use_quarantine
        extra_props = Dict(
            :quarantine_threshold => threshold,
            :quarantine_active => falses(3)
        )
    else
        extra_props = Dict()
    end
    Ns = [1000, 1000, 1000]
    β_und = [0.5, 0.5, 0.5]
    β_det = [0.05, 0.05, 0.05]
    rng = Xoshiro(42)
    C = 3
    migration_rates = zeros(C, C)
    for i in 1:C
        for j in 1:C
            migration_rates[i, j] = (Ns[i] + Ns[j]) / Ns[i]
        end
```

```
    end
    for i in 1:C
        migration_rates[i, :] ./= sum(migration_rates[i, :])
    end
    properties = Dict(
        :Ns => Ns,
        :β_und => β_und,
        :β_det => β_det,
        :migration_rates => migration_rates,
        :infection_period => 14,
        :detection_time => 7,
        :death_rate => 0.02,
        :reinfection_probability => 0.1,
        :C => C
    )
    if use_quarantine
        properties[:quarantine_threshold] = threshold
        properties[:quarantine_active] = falses(3)
        properties[:use_quarantine] = true
    else
        properties[:use_quarantine] = false
    end
    space = GraphSpace(complete_graph(C))
    model = StandardABM(Person, space; properties, rng, agent_step! = si
    for city in 1:C
        for _ in 1:Ns[city]
            add_agent!(city, model, 0, :S)
        end
```

```
    end
    city_agents = ids_in_position(1, model)
    infected_ids = sample(rng, city_agents, 10; replace=false)
    for id in infected_ids
        model[id].status = :I
        model[id].days_infected = 1
    end
    infected_vals = Float64[]
    for step in 1:100
        if use_quarantine
            for city in 1:3
                agents_city = [a for a in allagents(model) if a.pos == c
                total = length(agents_city)
                infected = count(a -> a.status == :I, agents_city)

                if total > 0 && infected / total > properties[:quarantin
                    properties[:quarantine_active][city] = true
                end
            end
        end
        for agent in allagents(model)
            if !(use_quarantine && properties[:quarantine_active][agent.
                current_city = agent.pos
                probs = properties[:migration_rates][current_city, :]
                target = sample(abmrng(model), 1:C, Weights(probs))
                if target != current_city
                    move_agent!(agent, target, model)
                end
```

```
            end
            if agent.status == :I
                rate = if agent.days_infected < properties[:detection_ti
                    properties[:β_und][agent.pos]
                else
                    properties[:β_det][agent.pos]
                end
                n_infections = rand(abmrng(model), Poisson(rate))
                if n_infections > 0
                    neighbors = [a for a in agents_in_position(agent.pos
                    shuffle!(abmrng(model), neighbors)
                    cnt = 0
                    for contact in neighbors
                        cnt >= n_infections && break
                        if contact.status == :S
                            contact.status = :I
                            contact.days_infected = 1
                            cnt += 1
                        elseif contact.status == :R && rand(abmrng(model
                            contact.status = :I
                            contact.days_infected = 1
                            cnt += 1
                        end
                    end
                end
                agent.days_infected += 1
            end
            if agent.status == :I && agent.days_infected >= properties[:
```

```
                if rand(abmrng(model)) <= properties[:death_rate]
                    remove_agent!(agent, model)
                else
                    agent.status = :R
                    agent.days_infected = 0
                end
            end
        end
        push!(infected_vals, infected_count(model))
    end
    return infected_vals
end
println("Запуск без карантина...")
infected_no = run_with_quarantine(false, 0.0)

println("Запуск с карантином (порог=10%)...")
infected_yes = run_with_quarantine(true, 0.1)

p5 = plot(infected_no, label="Без карантина", linewidth=2)
plot!(p5, infected_yes, label="Карантин (10%)", linewidth=2)
xlabel!(p5, "Дни")
ylabel!(p5, "Инфицированные")
title!(p5, "Задание 5: Эффективность карантина")
savefig(p5, plotsdir("task5.png"))

peak_no = maximum(infected_no)
peak_yes = maximum(infected_yes)
println("Без карантина: пик = $(round(peak_no)) человек")
```

```
println("С карантином: пик = $(round(peak_yes)) человек")
```

```
 задание 5
Запуск без карантина...
Запуск с карантином (порог=10%)...
Без карантина: пик = 3000.0 человек
С карантином: пик = 3000.0 человек
```

задание 6

```
println("\n задание 6")
function cost(x)
    peak_vals = Float64[]
    dead_vals = Float64[]
    for rep in 1:3
        model = initialize_sir(;
            Ns=[1000,1000,1000],
            β_und=fill(x[1],3),
            β_det=fill(x[1]/10,3),
            detection_time=round(Int,x[2]),
            death_rate=x[3],
            Is=[10,0,0],
            seed=42+rep
        )
        peak = 0.0
        for _ in 1:100
            Agents.step!(model,1)
            peak = max(peak, infected_count(model)/nagents(model))
```

```
        end
        push!(peak_vals, peak)
        push!(dead_vals, (3000 - nagents(model))/3000)
    end
    mean_peak = mean(peak_vals)
    mean_dead = mean(dead_vals)

    penalty = mean_peak > 0.3 ? 10.0 : 0.0

    return mean_dead + penalty
end
result = bboptimize(
    cost,
    SearchRange=[(0.1,0.8),(3.0,14.0),(0.001,0.05)],
    NumDimensions=3,
    MaxTime=60
)
best = best_candidate(result)
println("Оптимум: ", best)
@save datadir("opt.jld2") best
```

```
 задание 6
Starting optimization with optimizer DiffEvoOpt{FitPopulation{Float64},
0.00 secs, 0 evals, 0 steps
30.83 secs, 2 evals, 1 steps, fitness=10.232888889

Optimization stopped after 2 steps and 87.09 seconds
Termination reason: Max time (60.0 s) reached
```

```
Steps per second = 0.02
Function evals per second = 0.05
Improvements/step = NaN
Total function evaluations = 4


Best candidate found: [0.631435, 7.16132, 0.0036587]

Fitness: 10.024666667

Оптимум: [0.6314346433885847, 7.161321560143902, 0.0036587045439534457]
```

5 Выводы

В ходе проделанной работы ознакомилась с агентным подходом и реализацией основных моделей в агентном подходе на примере модели SIR.

Список литературы

- Лабораторная №4